Lab: Provision Your First Infrastructure on Azure with Terraform

Learning Objectives

By the end of this lab, you will be able to:

1. Write and execute Terraform scripts for Azure infrastructure.
2. Authenticate to Azure using the Azure CLI.
3. Create and configure an Azure Resource Group with Terraform.
4. Understand key Terraform commands and workflow.

Pre-requisites

Make sure you have the following before starting:

- A valid **Azure account** with an active subscription.
- **Terraform** installed locally.
- **Azure CLI** installed locally.
- **Visual Studio Code (VS Code)** or any IDE installed.

*If you haven't completed these, please go through the previous lab on **Terraform setup**.*

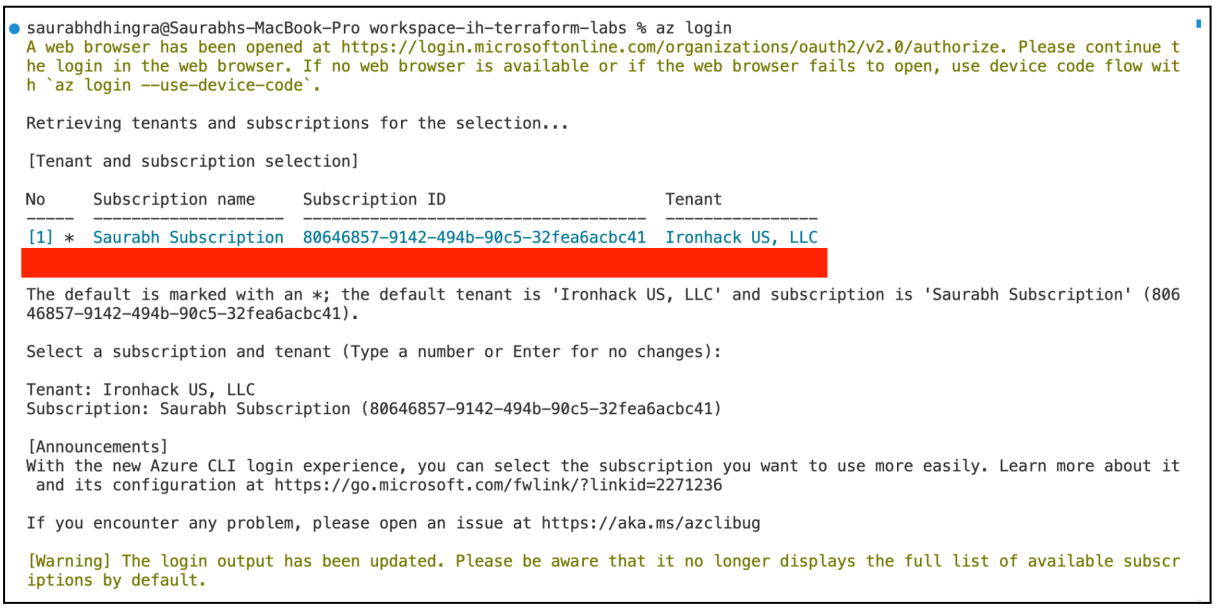
Authentication and Authorization

Terraform needs to authenticate with Azure to create resources. There are multiple ways to do this, but in this lab, we'll use the **Azure CLI** method.

Run the following command in your terminal:

```
az login
```

This will open a browser window where you can enter your Azure credentials.



If you have **multiple subscriptions**, select the one you want to use:

```
az account set --subscription "<your-subscription-id>"
```

To get your subscription ID, run:

```
az account list
```

Step 1: Create Terraform Configuration Files

1.1 Create Project Folder

In VS Code, create a new folder called **resource_group**. This will be your **Terraform root module**.

1.2 Configure the Provider

Create a file named **providers.tf** and add:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "4.26.0"
    }
  }
}

provider "azurerm" {
  features {}

  subscription_id = "<your-subscription-id>"
}
```

Explanation:

- **terraform block** → sets required providers and versions.
- **provider "azurerm"** → tells Terraform to use Azure.
- **features {}** → required by AzureRM provider (even if empty).
- **subscription_id** → tells Terraform which subscription to deploy to.

1.3 Initialize Terraform

Run:

```
terraform init
```

```
saurabhdhingra@Saurabhs-MacBook-Pro workspace-ih-terraform-labs % terraform init
Terraform initialized in an empty directory!

The directory has no Terraform configuration files. You may begin working
with Terraform immediately by creating Terraform configuration files.
saurabhdhingra@Saurabhs-MacBook-Pro workspace-ih-terraform-labs % cd lab1-first-resource
saurabhdhingra@Saurabhs-MacBook-Pro lab1-first-resource % terraform init
saurabhdhingra@Saurabhs-MacBook-Pro lab1-first-resource % cd
saurabhdhingra@Saurabhs-MacBook-Pro ~ % cd workspace-ih-terraform-labs
saurabhdhingra@Saurabhs-MacBook-Pro workspace-ih-terraform-labs % cd lab1_resource_group
saurabhdhingra@Saurabhs-MacBook-Pro lab1_resource_group % terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "4.26.0"...
- Installing hashicorp/azurerm v4.26.0...
- Installed hashicorp/azurerm v4.26.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
saurabhdhingra@Saurabhs-MacBook-Pro lab1_resource_group %
```

This will download the required Azure provider and create .terraform and lock files in your folder.

1.4 Define Variables

Create a file named **variables.tf**:

```
variable "resource_group_name" {
  type        = string
  description = "Name of the Resource Group"
}

variable "location" {
  type        = string
  description = "Azure region where resources will be created"
  default     = "West Europe"
}

variable "tags" {
  type        = map(string)
  description = "Tags to assign to resources"
}
```

1.5 Create Resource Group

Create a file named **resource_group.tf**:

```
resource "azurerm_resource_group" "resource_group" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}
```

1.6 Add Variable Values

Create a file named **terraform.tfvars**:

```
resource_group_name = "rg-terraform-vmlabs"
location            = "West Europe"
tags = {
  environment = "dev"
}
```

Step 2: Run Terraform Commands

Format your code

```
terraform fmt
```

1. (Makes the code clean and readable.)

Validate configuration

```
terraform validate
```

2. (Checks syntax and catches errors.)

Preview changes

```
terraform plan
```

3. (Shows what Terraform will create in Azure.)

```
saurabhdhingra@Saurabhs-MacBook-Pro lab1_resource_group % terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_resource_group.resource_group will be created
  + resource "azurerm_resource_group" "resource_group" {
      + id       = (known after apply)
      + location = "westeurope"
      + name     = "rg-terraform-vmlabs"
      + tags     = {
          + "environment" = "dev"
        }
    }

Plan: 1 to add, 0 to change, 0 to destroy.

─────────────────────────────────────────────────────────────────────────────
```

Apply configuration

```
terraform apply --auto-approve
```

4. (Actually provisions the Resource Group in Azure.)

```
saurabhdhingra@Saurabhs-MacBook-Pro lab1_resource_group % terraform apply --auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_resource_group.resource_group will be created
  + resource "azurerm_resource_group" "resource_group" {
      + id       = (known after apply)
      + location = "westeurope"
      + name     = "rg-terraform-vmlabs"
      + tags     = {
          + "environment" = "dev"
        }
    }

Plan: 1 to add, 0 to change, 0 to destroy.
azurerm_resource_group.resource_group: Creating...
azurerm_resource_group.resource_group: Still creating... [10s elapsed]
azurerm_resource_group.resource_group: Creation complete after 16s [id=/subscriptions/80646857-9142-494b-90c5-32fea6acbc4
1/resourceGroups/rg-terraform-vmlabs]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

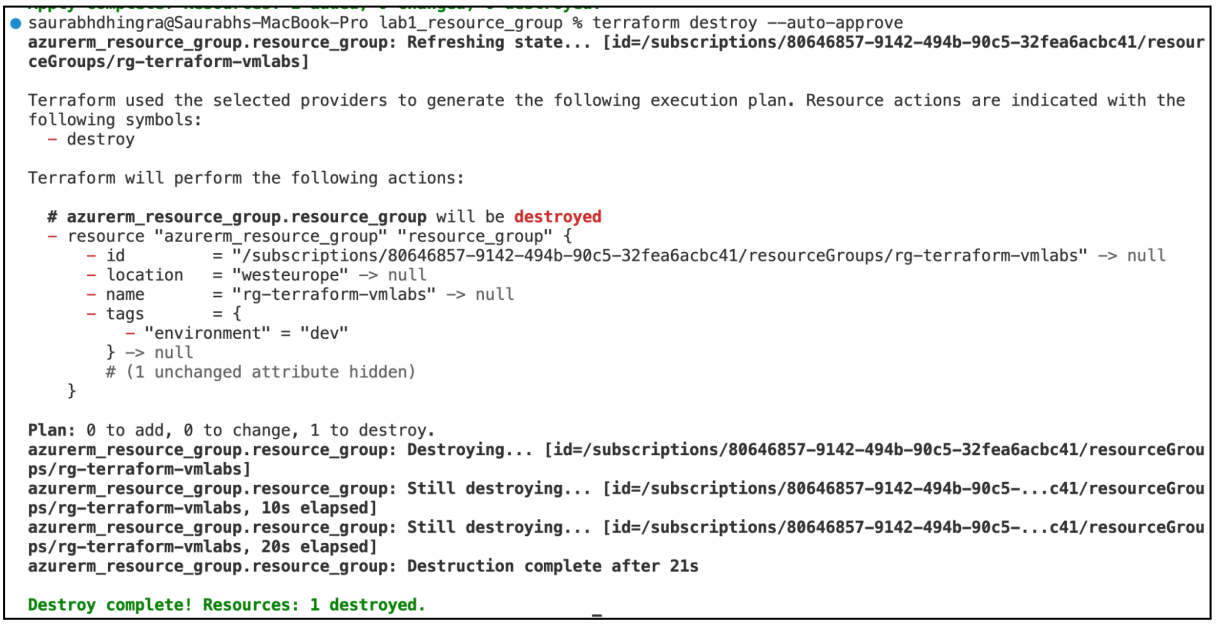
5. **Verify in Azure Portal**

Log in to Azure Portal, navigate to **Resource Groups**, and confirm your new `rg-terraform-vmlabs` exists.

Step 3: Clean Up (Destroy Resources)

To avoid unnecessary costs, delete the Resource Group by running:

```
terraform destroy --auto-approve
```



Congratulations!

You've just provisioned your **first Azure resource** (a Resource Group) using Terraform.

You now know how to:

- Authenticate Terraform with Azure.
- Write provider and resource blocks.
- Use variables and tfvars files.
- Run Terraform's core workflow (init → plan → apply → destroy).

This sets the foundation for provisioning more complex Azure infrastructure with Terraform.